Implementación y demostración visual de un monitor productor–consumidor mediante mutex y variables de condición

Equipo del Proyecto 7: completar integrantes
Sistemas Operativos
Completar institución, grupo y docente

***Resumen*—Se presenta una implementación en C17 del problema productor–consumidor con búfer limitado, utilizando un mutex y dos variables de condición POSIX, sin semáforos. El monitor encapsula un búfer circular FIFO, sus invariantes y el protocolo de cierre. Dos escenarios guiados evidencian el bloqueo real del productor por falta de espacio y del consumidor por ausencia de datos. Una interfaz de terminal a color representa snapshots consistentes y eventos reales sin mantener el mutex durante el renderizado. Se verificaron orden FIFO, límites, cierre durante esperas, fallos de creación de hilos, controles de terminal y pruebas de estrés, además de ejecuciones con sanitizadores. Se describen los límites de la evidencia experimental y los entregables reproducibles.**

***Index Terms*—monitor, exclusión mutua, POSIX Threads, variable de condición, productor–consumidor.**

I. Introducción

La concurrencia permite que varias actividades progresen sobre recursos compartidos, pero introduce intercalaciones que pueden invalidar supuestos de una ejecución secuencial. En un búfer limitado, una inserción sobre memoria aún ocupada pierde información y una extracción sin datos produce un resultado inválido. El problema requiere tanto exclusión mutua como coordinación entre productor y consumidor.

El objetivo del proyecto es construir una solución correcta y una demostración comprensible. No basta con dibujar un hilo detenido: deben observarse operaciones reales de espera y reanudación. La implementación distingue el estado sincronizado del recurso, el ciclo de vida de los hilos y la presentación en terminal.

II. Fundamentos

II-A. Monitor y encapsulación

El trabajo clásico de Hoare desarrolla el monitor como concepto para estructurar sistemas operativos [1]. En este proyecto se adopta la idea de reunir estado compartido, operaciones controladas y sincronización en una abstracción. C no incorpora un monitor como construcción sintáctica: se implementa mediante un tipo opaco y funciones que respetan una disciplina de acceso.

II-B. Mutex y espera por condición

El mutex protege las modificaciones y observaciones del estado. Una condición se asocia a un predicado: disponer de espacio o disponer de datos. La llamada `pthread_cond_wait` libera el mutex durante la espera y lo recupera al regresar. El retorno no certifica la verdad del predicado, por lo que se reevalúa en un ciclo `while` [2]. Esto cubre despertares espurios y cambios del estado antes de recuperar la exclusión.

Una señal no es un permiso almacenado ni una transferencia inmediata de propiedad del mutex. El hilo despertado debe competir por adquirirlo [3]. La implementación emplea semántica de tipo Mesa: el que señala continúa hasta liberar la exclusión. Por tanto, se distingue entre notificación, retorno de espera y operación efectivamente completada.

II-C. Elección de C17

C17 incorpora correcciones y aclaraciones sin introducir grandes cambios respecto a C11 [4]. Se selecciona explícitamente con `-std=c17` para evitar depender del dialecto por defecto del compilador. POSIX Threads es la API de sincronización exigida por el proyecto, no una característica exclusiva de esta versión del lenguaje. Elegir un estándar más reciente no elimina la necesidad de razonar sobre exclusión y predicados.

III. Diseño de la solución

III-A. Estructura y alcance

El programa consta de un productor, un consumidor y un hilo principal. Este último procesa teclado, coordina el escenario y dibuja snapshots. El módulo `monitor.c` posee un mutex, `not_full`, `not_empty`, un arreglo de enteros, índices circulares y contadores. Su representación no es accesible mediante el encabezado público.

La capacidad configurable se limita a 1–16 elementos para mantener la interfaz legible. La cantidad de elementos puede alcanzar un millón. Estas restricciones pertenecen a la demostración, no a las primitivas POSIX.

III-B. Invariantes

Sea N la capacidad, C la ocupación, P el total insertado y K el total extraído. Se mantienen:

$$0 \le C \le N, \tag{1}$$

$$0 \le head, tail < N, \tag{2}$$

$$P - K = C. \tag{3}$$

La inserción escribe en $tail$, lo avanza módulo N e incrementa C y P. La extracción lee en $head$, lo avanza y actualiza C y K. Todos esos pasos ocurren bajo el mismo mutex. El orden de lectura reproduce el orden de inserción. Las aserciones del monitor verifican límites y conservación de elementos.

III-C. Operaciones

El siguiente pseudocódigo resume el productor; la implementación agrega manejo de errores, eventos y contadores.

```
lock(mutex);
while (count == capacity && !closed)
   cond_wait(not_full, mutex);
if (closed) { unlock(mutex); return false; }
insert_fifo(value);
cond_signal(not_empty);
unlock(mutex);
```

Para el consumidor:

```
lock(mutex);
while (count == 0 && !closed)
   cond_wait(not_empty, mutex);
if (count == 0) { unlock(mutex); return false; }
value = remove_fifo();
cond_signal(not_full);
unlock(mutex);
```

El estado compartido constituye la memoria de lo ocurrido: aun si no había un hilo esperando al emitir una señal, una operación posterior consulta el predicado antes de decidir esperar.

III-D. Terminación y errores

Cuando termina el productor se marca el monitor como cerrado. Se despiertan ambas condiciones mediante `broadcast`, de modo que todo hilo en espera pueda revisar el cierre. Se rechazan nuevas inserciones, pero el consumidor puede drenar lo pendiente. El principal realiza `pthread_join` antes de destruir condiciones, mutex y memoria.

La cancelación solicitada con teclado o señales es cooperativa. Los manejadores solo asignan una bandera compatible con señales; el flujo normal ejecuta la limpieza. Atómicos separados controlan ritmo, parada y el indicador de integridad; no sincronizan el búfer. Los fallos de inicialización y creación de hilos se reportan y limpian. Un fallo interno inesperado de pthread provoca terminación explícita, evitando continuar con datos sin protección.

IV. Demostración visual

IV-A. Bloqueos observables

En el escenario del productor se crea únicamente ese trabajador. Tras llenar el búfer, intenta insertar otro elemento y espera. El consumidor no existe todavía, por lo que no puede liberar espacio prematuramente. En el escenario del consumidor se inicia solo ese hilo sobre un búfer vacío.

La transición a espera se registra bajo el mutex antes de llamar a `cond_wait`. El principal consulta el estado adquiriendo el mismo mutex: para observarlo, el trabajador tuvo que liberarlo al esperar. Una vez observada la espera, Enter crea la contraparte. La demostración no supone que cierto número de milisegundos baste para establecer el orden de planificación.

IV-B. Interfaz y trazabilidad

Se emplean ANSI y `termios`, sin dependencias externas de desarrollo. Cian identifica al productor, magenta al consumidor, amarillo a la espera y verde a los resultados. Las etiquetas acompañan los colores para facilitar lectura. La vista principal muestra tarjetas de los hilos, celdas y tres etapas de la demo. La tecla D abre índices, estadísticas y eventos numerados, evitando saturar la pantalla.

Cada evento registra tiempo monotónico, tipo, actor, valor y ocupación. El historial es un anillo de 128 entradas. La copia del estado se realiza bajo el mutex; dibujo, teclado y pausas ocurren fuera de él. La pantalla no afirma cuál hilo posee el mutex en el instante del renderizado. Los eventos son históricos y los snapshots pueden quedar desactualizados antes de aparecer en pantalla.

La evidencia buscada es WAIT, operación de la contraparte, SIGNAL, WAKE y operación reanudada. El modo textual anuncia huecos si el anillo sobrescribe eventos. A alta velocidad no se garantiza visualizar cada operación; la verificación del orden de todos los elementos permanece dentro del consumidor.

V. Metodología y resultados

Se ejecutaron pruebas el 10 de octubre de 2026 en Ubuntu 26.04 LTS con GCC 15.2.0. La compilación utiliza advertencias estrictas tratadas como errores. Las pruebas unitarias cubren FIFO, 100 ciclos de wraparound, capacidad uno, ambos bloqueos, cierre idempotente, drenaje y cierre durante espera.

La suite de integración contiene seis métodos con subcasos de argumentos, secuencias de señalización, estrés y errores. En estrés se transfieren 10 000 valores para cada combinación de cuatro capacidades y cuatro repeticiones: 160 000 transferencias por ejecución de esa suite. Una variante enlazada con un wrapper provoca fallos controlados de la primera y segunda creación de hilos en los tres escenarios.

Seis métodos de pseudoterminal verifican demos guiadas, paleta, velocidad, continuación, redimensionamiento, salida por teclado/señales y restauración exacta de los atributos de terminal. Los subcasos incluyen ambas clases de espera, alternancia de detalles, locales UTF-8/ASCII y búferes densos en terminales pequeñas.

Cuadro I
Validación local observada

Comando	Resultado
`make check`	Unitarias e integración/terminal aprobadas.
`make sanitize`	Suite aprobada con ASan y UBSan.
`make tsan`	Unitarias y 10 000 transferencias aprobadas.

LeakSanitizer no pudo ejecutarse inicialmente bajo el trazador del sandbox. Se repitió fuera de ese entorno y pasó sin desactivar detección de fugas. La primera prueba ad hoc de

colores heredó `NO_COLOR`; la suite definitiva controla explícitamente esa variable y comprueba ambas configuraciones.

No se interpreta el éxito de las pruebas como demostración exhaustiva de todas las intercalaciones. No se midieron equidad, rendimiento ni latencia máxima, y no se evaluó una configuración con múltiples productores o consumidores. Los resultados son locales; el workflow remoto se configura como verificación futura.

VI. Conclusiones

La solución separa exclusión mutua, espera por predicados y observación visual. El monitor conserva los invariantes del búfer y permite un cierre explícito sin semáforos. La creación diferida de la contraparte hace reproducibles las dos situaciones exigidas sin utilizar demoras como mecanismo de corrección. La terminal complementa la explicación teórica con estados y eventos reales, mientras las pruebas aportan evidencia de funcionalidad, limpieza y orden de datos.

Referencias

[1] C. A. R. Hoare, "Monitors: An Operating System Structuring Concept," *Communications of the ACM*, vol. 17, no. 10, pp. 549–557, 1974, doi: 10.1145/355620.361161.

[2] The Open Group, "pthread_cond_clockwait, pthread_cond_wait," *POSIX.1-2024*. [Online]. Available: https://pubs.opengroup.org/onlinepubs/9799919799/functions/pthread_cond_clockwait.html

[3] The Open Group, "pthread_cond_broadcast, pthread_cond_signal," *POSIX.1-2017*, 2018 edition. [Online]. Available: https://pubs.opengroup.org/onlinepubs/9699919799/functions/pthread_cond_broadcast.html

[4] GNU Project, "C Standards Support in GCC." [Online]. Available: https://gcc.gnu.org/projects/c-status.html